NoSQL Databases

Week 6 - Topic 4 – Lesson 11 and 12

Key–Value • Document • Column-Family • Graph

SQL vs NoSQL • MongoDB schema design • SQL→NoSQL conversion

Outline

- Key-value databases
- Document databases
- Column-family (wide-column) databases
- Graph databases
- Comparison of SQL vs. NoSQL
- MongoDB and schema design
- SQL-to-NoSQL schema conversion

What is NoSQL?

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

Definition

- A family of "non-relational" databases with data models optimized for flexibility, scalability, and specific access patterns.
- NoSQL databases handle large volumes of unstructured and semi-structured data, offering the scalability and flexibility today's diverse workloads demand.

Why teams choose it

- Scale-out (sharding/partitioning)
- Schema-flexible data (JSON-like)
- High write throughput / low latency
- Specialized queries (graphs, time series)

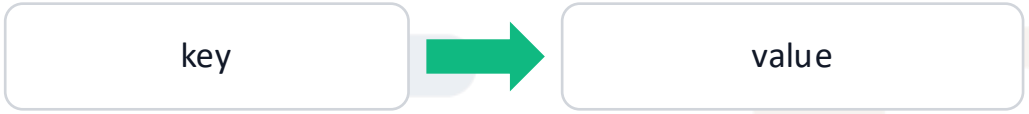
Common trade-offs

- Fewer joins; denormalization is common
- Consistency/transaction semantics vary
- Different query languages & APIs
- Operational concerns (partition keys, indexes)

Rule of thumb: start from your access patterns (reads/writes), not from entity-relationship diagrams.

NoSQL data models

Key–Value



Fast lookups by primary key

Column-family (wide-column)

rowKey	colA	colB	...

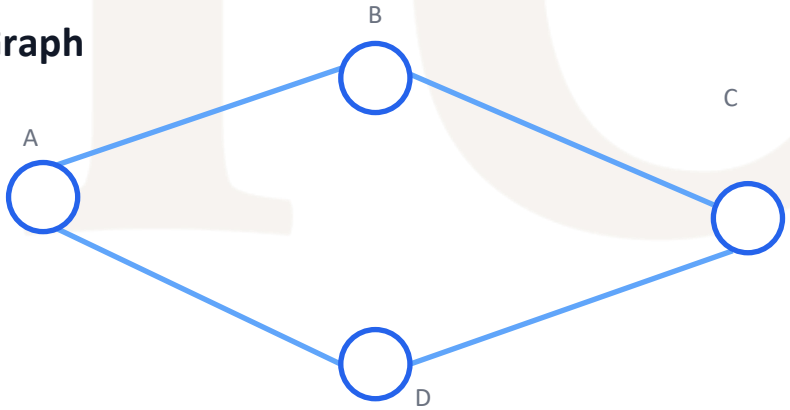
Sparse / wide rows

Document

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```

Nested, JSON-like records

Graph



Relationships are first-class

Key–value databases

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

How they store data

Data is a collection of key → value pairs.
The key is unique; the value is opaque to the store (often a blob or JSON).

Key: user:12345
Value: {"name": "foo bar", "email": "foo@bar.com", "designation": "software developer"}

```
GET key
PUT key value
DEL key
```

Where they shine

- Caching & session state
- Feature flags, counters, rate limits
- User profiles by ID
- High-throughput event ingestion

Examples

MongoDB
DynamoDB
Aerospike

Document databases

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

Core idea

Store records as self-describing documents (JSON/BSON). Documents can be nested and vary in shape — while still supporting indexes and rich queries.

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```

Use cases

- Content management & catalogs
- User-facing apps with evolving requirements
- Event data and telemetry
- Microservices with bounded contexts

Examples

MongoDB
Couchbase
Cosmos DB

Column-family databases

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

Mental model

Think: “partitioned tables” where each row can be very wide (and columns can be sparse). Data is distributed by a partition key.

partition_key	time	metric	value
user#42	2026-01-29	clicks	17
user#42	2026-01-29	purchases	2
...			

Design principle

Model around queries.

- Choose a partition key that matches the primary access path
- Prefer predictable queries over ad-hoc joins
- Denormalize to avoid cross-partition reads
- Use clustering / sort keys for time-series & range scans

Examples:
Cassandra
Hbase
Bigtable

Note - Wide-column stores excel at massive scale with predictable query patterns

Graph databases

Data is stored as:

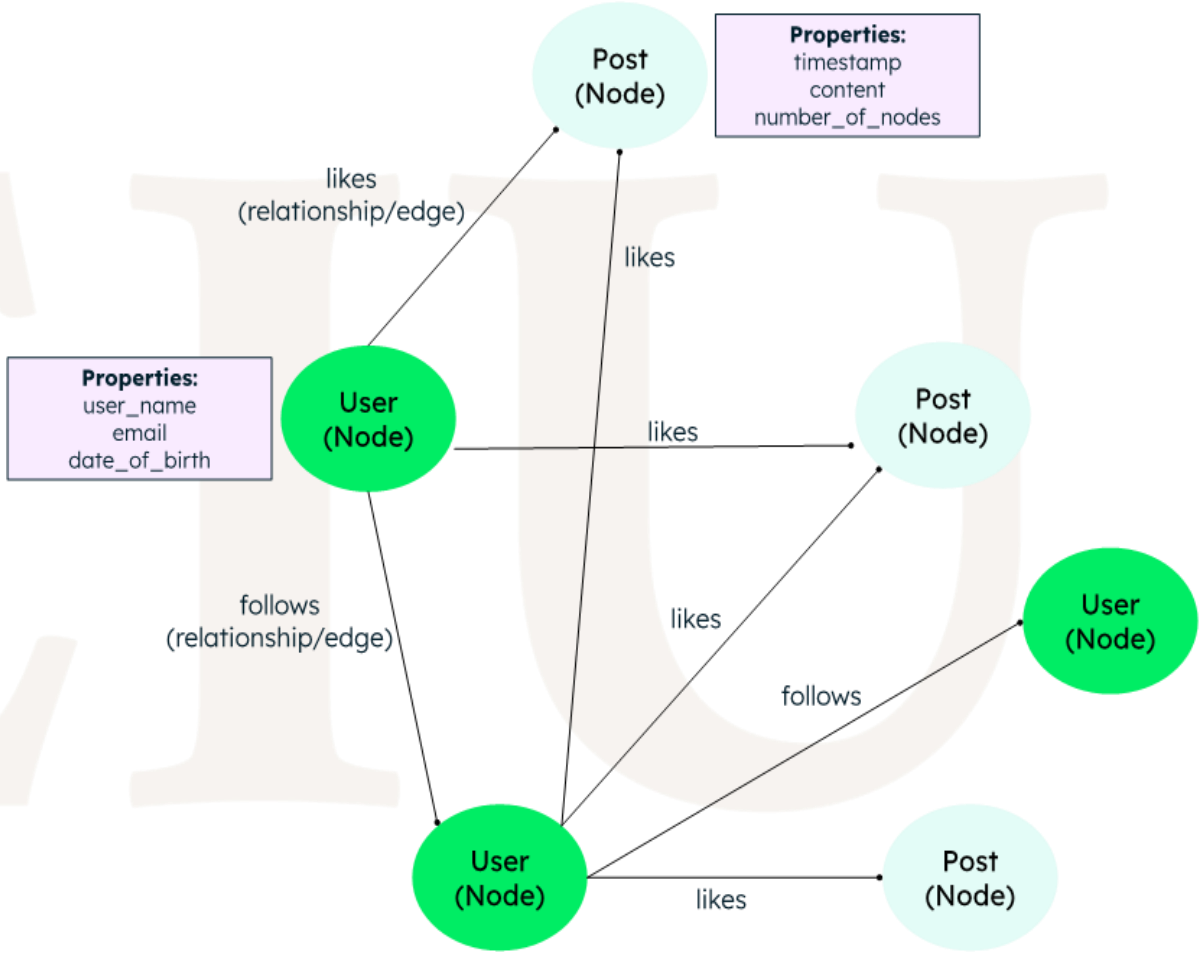
- Nodes (entities)
- Relationships (edges) with direction
- Properties on both nodes and relationships
- Nodes typically store information about people, places, and things (like nouns), while edges store information about the relationships between the nodes.

When graphs win

- Deep relationship traversals
- Recommendation & fraud graphs
- Network / IT topology
- Knowledge graphs & metadata

Examples:

- Neo4j
- Neptune
- JanusGraph



Note - Graph databases make relationships explicit, enabling fast traversals and pattern matching

Comparison: SQL vs NoSQL

Dimension	SQL (Relational)	NoSQL (Non-relational)
Schema	Predefined / migrations	Flexible (often “schema-on-read”)
Data model	Tables + relations	Documents / key-values / graphs / columns
Joins	Native joins & constraints	Often denormalize; app-side joins
Scaling	Commonly vertical + replicas	Commonly horizontal partitioning
Transactions	ACID is standard	Varies: some support ACID, many favor availability
Best for	Complex queries + integrity	High scale + evolving models

Note: Many modern systems are “multi-model” (e.g., SQL with JSON, or NoSQL with transactions).

Why Need NoSQL

- Big Data Applications: Efficiently stores and processes massive amounts of unstructured and semi-structured data.
- Real-Time Analytics: Supports fast queries and analysis for use cases like recommendation engines or fraud detection.
- Scalable Web Applications: Handles high traffic and large user bases by scaling horizontally across servers.
- Flexible Data Storage: Manages diverse data formats (JSON, key-value, documents, graphs) without rigid schemas.

Relational Database

User table

ID	first_name	last_name	cell	city
1	Leslie	Yepp	8125552344	Pawnee

Hobbies table

ID	user_id	hobby
10	1	scrapbooking
11	1	eating waffles
12	1	working

MongoDB

```
{
   "_id": 1,
   "first_name": "Leslie",
   "last_name": "Yepp",
   "cell": "8125552344",
   "city": "Pawnee",
   "hobbies": ["scrapbooking", "eating
waffles", "working"]
}
```

- No need for joins
- No need for data normalization

Why Need NoSQL....

Key-value database	Document database	Wide-column database	Graph database
Basic support for data visualization	Basic support for data visualization	Basic support for data visualization	Basic support for data visualization
Basic authentication, limited access control	Role-based access control, encryption at-rest, encryption in transit, and in use	Role-based access control (RBAC), encryption at rest	Role-based access control (RBAC), encryption at rest, in transit
Limited support for online data archival	Automatic online data archival	Limited support for online data archival	Limited support for online data archival
Basic key-based data retrieval	Full-text search, vector search	Limited search capabilities	Limited search capabilities
Basic CRUD operations, limited data manipulation capabilities	Advanced query and data manipulation capabilities	Data manipulation and CRUD capabilities	Supports advanced graph traversal and manipulation operations
Stores any type of data	Stores JSON/BSON data	Columns and rows matrix	Nodes, edges, and relationships
Limited support for complex analytics	Suited for time-series, IoT analytics, real-time analytics	Suited for time-series data, IoT analytics	Well-suited for graph analytics
Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency	Horizontal scalability and eventual consistency
Simple indexes	Indexes on fields	Secondary indexes	Indexes on nodes/edges
Limited by key	Rich querying capability	Limited by columns	Specialized graph query
Schemaless	Flexible schema	Flexible schema	Flexible schema
Simple key-value pairs	JSON/BSON documents	Column and row families	Nodes, edges, and relationships

MongoDB and schema design

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

MongoDB

MongoDB = document database (BSON).

Design around "document boundaries"

- A single document is the unit of atomic update
- Embed data that is read/written together
- Reference data that grows without bound or is shared widely
- Add indexes to match query predicates and sort orders

Goal: satisfy the most common query in 1–2 reads.

Schema flexibility ≠ no schema

- Define required fields and types at the application layer
- Use schema validation when appropriate
- Evolve schema via migrations (add fields, backfill, dual-read)
- Watch document size growth (16MB document limit)

Practices for MongoDB Schema Design

- Embedding Documents
- Using References
- Denormalizing Data
- Indexing
- Partitioning

MongoDB: embed vs reference

Embed when…

- One-to-few relationships
- Data is frequently read together
- You need atomic updates across related fields
- Bounded growth (document won’t explode)

```
// Embedded
{ _id: "u1",
  name: "Amina",
  addresses: [ {city:"Lagos"}, {city:"Accra"} ]
}
```

Reference when…

- One-to-many with unbounded growth
- Many-to-many sharing
- Sub-entities are updated independently
- You need separate lifecycle / permissions

```
// Referenced
users:   { _id:"u1", name:"Amina" }
addresses:{ _id:"a9", userId:"u1",
city:"Lagos" }
```

SQL → NoSQL schema conversion

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

A practical, repeatable process

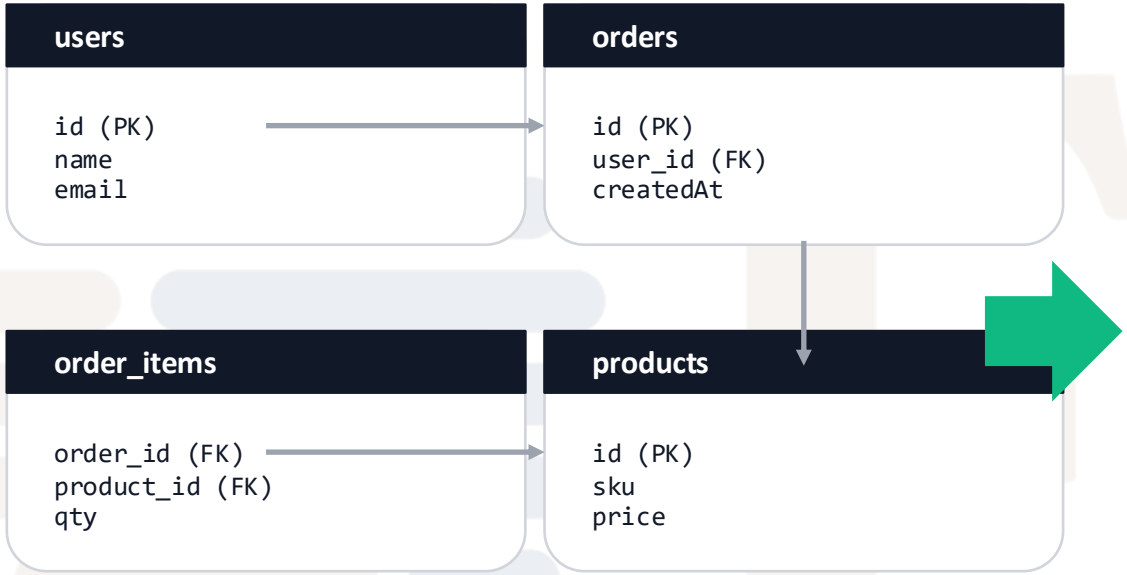
1. List the critical queries (read paths) and write paths
2. Choose a target NoSQL family (document / wide-column / graph / key-value)
3. Define an “aggregate” per query: what should be fetched together?
4. Replace joins with embedding, precomputed views, or references
5. Pick partition keys and indexes that match access patterns
6. Plan migration: dual-write or backfill + cutover
7. Add validation, monitoring, and correctness checks

The biggest conceptual change: model for queries, not for minimizing duplication

Example: relational e-commerce → MongoDB documents

EIU
TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG
EASTERN INTERNATIONAL UNIVERSITY

Relational (normalized)



MongoDB (document)

Common query: “fetch an order with its line items”

```
// orders collection
{
  _id: "o1001",
  userId: "u1",
  createdAt: ISODate("..."),
  items: [
    { productId: "p9", qty: 2, price:
19.99 },
    { productId: "p4", qty: 1, price:
5.49 }
  ]
}
```

Embedding eliminates a join for this read path.

Note - Conversion strategy: embed the “many” side when reads want the whole aggregate

Further reading and practice

- MongoDB - https://www.mongodb.com/resources/basics/databases/nosql-explained
- Introduction to NoSqL - https://www.geeksforgeeks.org/dbms/introduction-to-nosql/
- Design Your Schema - https://www.mongodb.com/docs/manual/data-modeling/schema-design-process/
- MongoDB Schema Design Best Practices and Techniques - https://www.geeksforgeeks.org/mongodb/mongodb-schema-design-best-practices-and-techniques/

- Software – MongoDB and Neo4J